Networking Fundamentals

Networking Overview Course · Module 1



The OSI model: 7 layers

A reference model — each layer serves the one above and relies on the one below

7	**Application**	Interface for applications	HTTP, DNS, SMTP
6	**Presentation**	Format, encrypt, compress	TLS, JPEG, UTF-8
5	**Session**	Open and manage sessions	RPC, NetBIOS
4	**Transport**	End-to-end delivery, ports	TCP, UDP
3	**Network**	Logical addressing, routing	IP, ICMP
2	**Data Link**	Local delivery by MAC	Ethernet, Wi-Fi
1	**Physical**	Bits on the medium	Copper, fiber, radio



Why layers?

Each layer can change independently — Wi-Fi can replace Ethernet without touching HTTP. Layers also give us a shared vocabulary for troubleshooting.



Remember it

Top down (7→1):
All People Seem To Need Data Processing
Bottom up (1→7):
Please Do Not Throw Sausage Pizza Away

TCP/IP: the model the Internet runs on

Four practical layers that map onto OSI's seven

OSI	TCP/IP	Key protocols
7 Application	Application	HTTP/HTTPS, DNS, DHCP, SSH, SMTP, FTP
6 Presentation		
5 Session		
4 Transport	Transport	TCP (reliable) · UDP (fast)
3 Network	Internet	IPv4, IPv6, ICMP
2 Data Link	Network Access	Ethernet, Wi-Fi (802.11), ARP
1 Physical		

Encapsulation: wrapping data layer by layer

Each layer adds its own header on the way down; the receiver removes them on the way up

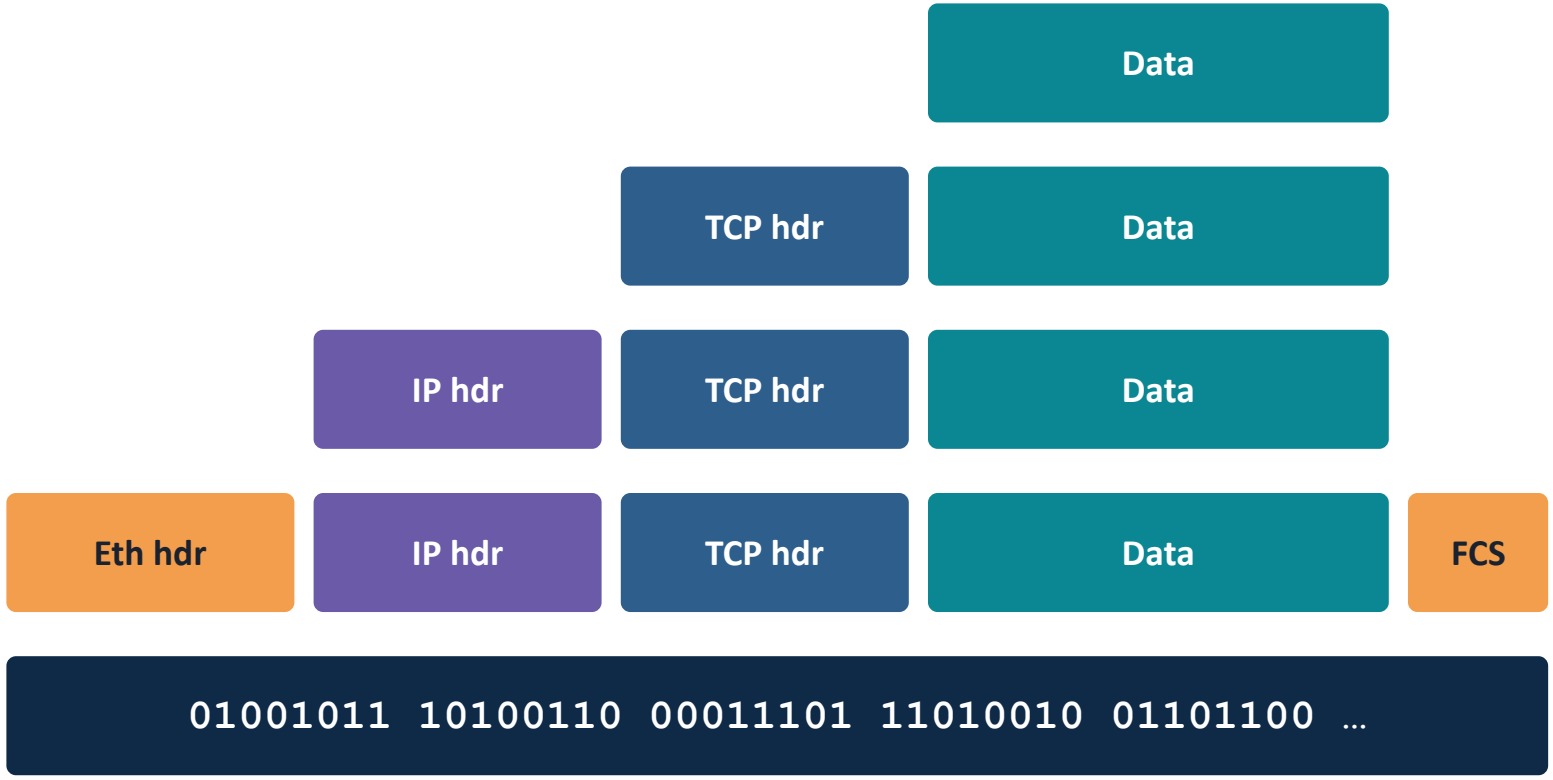
Application
PDU: Data

Transport
PDU: Segment

Network
PDU: Packet

Data Link
PDU: Frame

Physical
PDU: Bits



FCS = Frame Check Sequence, a checksum the receiver uses to detect corrupted frames.

IP addresses: who is talking to whom

A logical address that identifies a host and the network it belongs to



IPv4 = 32 bits, written as 4 octets of 8 bits · each octet is 0–255

Private ranges (RFC 1918)

- `10.0.0.0/8`
- `172.16.0.0/12`
- `192.168.0.0/16`
- Not routed on the Internet — reached through NAT

Special addresses

- 127.0.0.1 — loopback (this host)
- 169.254.x.x — APIPA, no DHCP reply
- 255.255.255.255 — broadcast
- 0.0.0.0 — 'any' / unknown

IPv6

- 128 bits, 8 groups of hex
- `2001:db8::1`
- '::' compresses runs of zeros
- ≈3.4 × 10^{38} addresses — no NAT needed

Subnets: splitting the address in two

The subnet mask (or /prefix) marks which bits are the network and which are the host

`192.168.10.0 /24`

Network: 192.168.10	Host: .0–.255
24 network bits	8 host bits

`Mask 255.255.255.0 = 11111111.11111111.11111111.00000000`

Example: split 192.168.10.0/24 into /26

2 bits borrowed → 4 subnets × 64 addresses (62 usable hosts)

.0/26	.64/26	.128/26	.192/26

Usable hosts = $2^h - 2$ (network + broadcast addresses are reserved)

Prefix	Subnet mask	Addresses	Usable
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/30	255.255.255.252	4	2



Same subnet? Deliver directly.
Different subnet? Send to the default gateway.

MAC addresses: the hardware identity

A 48-bit address assigned to every network interface, used for delivery on the local link

`00:1A:2B` : `3C:4D:5E`

OUI
Vendor ID (Intel, Cisco…)

Device ID
Unique per interface

48 bits = 12 hex digits
Written as 00:1A:2B:3C:4D:5E or 001A.2B3C.4D5E

Local only (Layer 2)
Never crosses a router — rewritten at every hop

Broadcast FF:FF:FF:FF:FF:FF
Reaches every device on the local segment

ARP: finding a MAC from an IP

Request · broadcast
“Who has 192.168.1.1? Tell 192.168.1.10”

Reply · unicast
“192.168.1.1 is at 00:1A:2B:3C:4D:5E”

Answers are cached in the ARP table — view it with `arp -a`

Switches, routers and gateways

Switches connect devices within a network; routers connect networks to each other



Switch

Layer 2

Connects devices in the same LAN

- Learns which MAC lives on which port
- Forwards frames only to the right port
- Floods broadcasts and unknown MACs

Uses: MAC address table



Router

Layer 3

Connects different IP networks

- Reads destination IP address
- Picks the best path from its routing table
- Separates broadcast domains

Uses: routing table



Default gateway

A router role

The exit door from your subnet

- Router interface on your subnet, e.g. 192.168.1.1
- Hosts send all off-subnet traffic here
- Wrong gateway = only local traffic works

Set by: DHCP or static config

A packet's journey across networks

IP addresses stay the same end to end; MAC addresses change at every hop



1 Local or remote?

PC A compares 93.184.215.14 with its own IP and mask — it is not on 192.168.1.0/24.

2 Send to the gateway

PC A ARPs for 192.168.1.1 and sends the frame to the router's MAC. The switch forwards it.

3 Route hop by hop

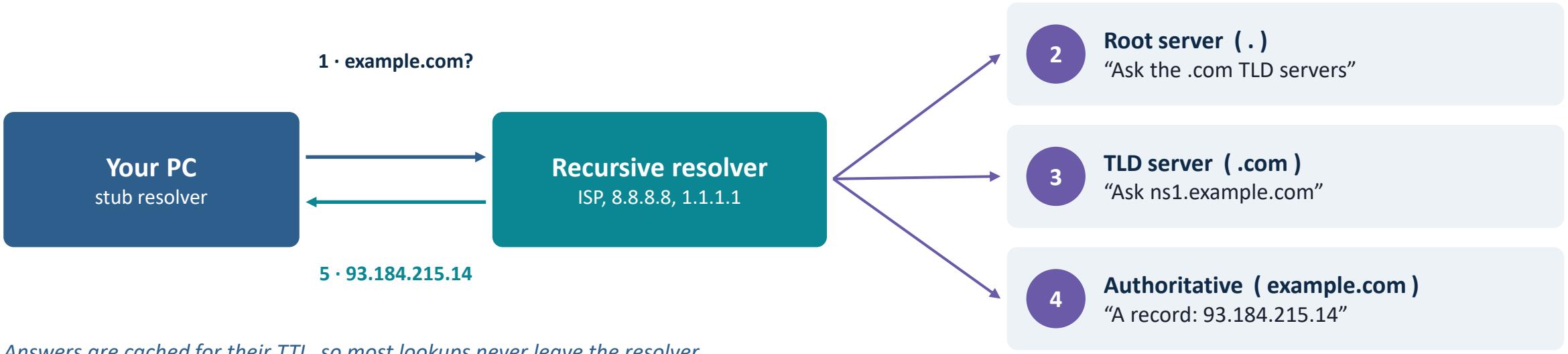
Each router strips the frame, reads the destination IP, looks up the next hop and builds a new frame.

4 Arrive and reply

The server receives the packet and replies to 192.168.1.10 the same way, in reverse.

DNS: the Internet's phone book

Translates names people remember into IP addresses computers use



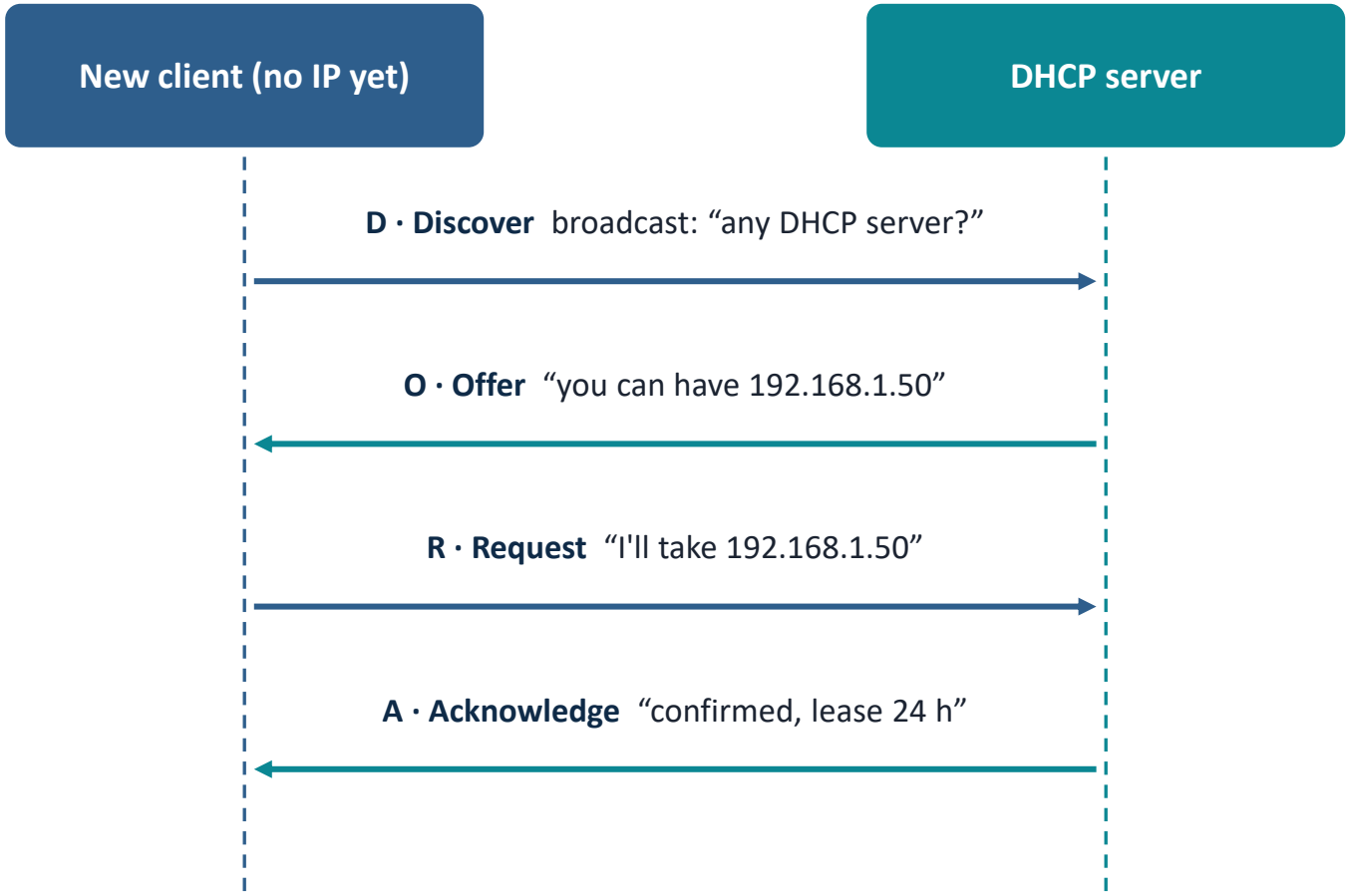
Answers are cached for their TTL, so most lookups never leave the resolver.

Common record types

A	AAAA	CNAME	MX	NS	PTR
Name → IPv4	Name → IPv6	Alias → name	Mail server	Name servers	IP → name

DHCP: automatic network configuration

A new device gets its settings in four messages — remember “DORA”



What the client receives

IP address	`192.168.1.50`
Subnet mask	`255.255.255.0`
Default gateway	`192.168.1.1`
DNS server	`192.168.1.1`
Lease time	renewed at 50%

Runs over UDP — server port 67, client port 68

Ports: many services on one host

The IP address finds the host; the port number finds the application

93.184.215.14 : 443

socket = IP address + port

0–1023	1024–49151	49152–65535
Well-known	Registered	Dynamic / client

TCP

- Connection-oriented (3-way handshake)
- Reliable, ordered, retransmits
- Web, email, SSH, file transfer

UDP

- Connectionless — just send
- No guarantees, low overhead
- DNS, DHCP, VoIP, video, gaming

Port	Service	Protocol
20/21	FTP	TCP
22	SSH	TCP
23	Telnet	TCP
25	SMTP	TCP
53	DNS	UDP/TCP
67/68	DHCP	UDP
80	HTTP	TCP
110	POP3	TCP
143	IMAP	TCP
443	HTTPS	TCP
3389	RDP	TCP

Demo: ports, sockets and the TCP handshake

See which ports are open, connect two programs and watch SYN, SYN-ACK, ACK

```
debian@lab: ~

# Which programs listen on which ports? (t=TCP u=UDP)
sudo ss -tulpn
# Port numbers <-> service names
getent services ssh domain https
# Open connections right now: local and remote sockets
ss -tn state established

# Terminal 1: capture, then listen on TCP port 9000
sudo tcpdump -nn -i lo tcp port 9000 &
nc -l 9000
# Terminal 2: connect and type a message
nc 127.0.0.1 9000

# Scan ports (only hosts you own, or scanme.nmap.org)
nmap -sT localhost
nmap -F scanme.nmap.org
```

Install

`netcat-openbsd nmap tcpdump`

What to look for

- The handshake: [S], then [S.], then [.] in tcpdump
- The client side gets a random high (ephemeral) port
- ss shows each connection as local IP:port ↔ remote IP:port

Connection-oriented vs connectionless

TCP sets up a conversation before it talks; UDP just sends



TCP: a phone call
Dial, confirm, talk in order, hang up

vs

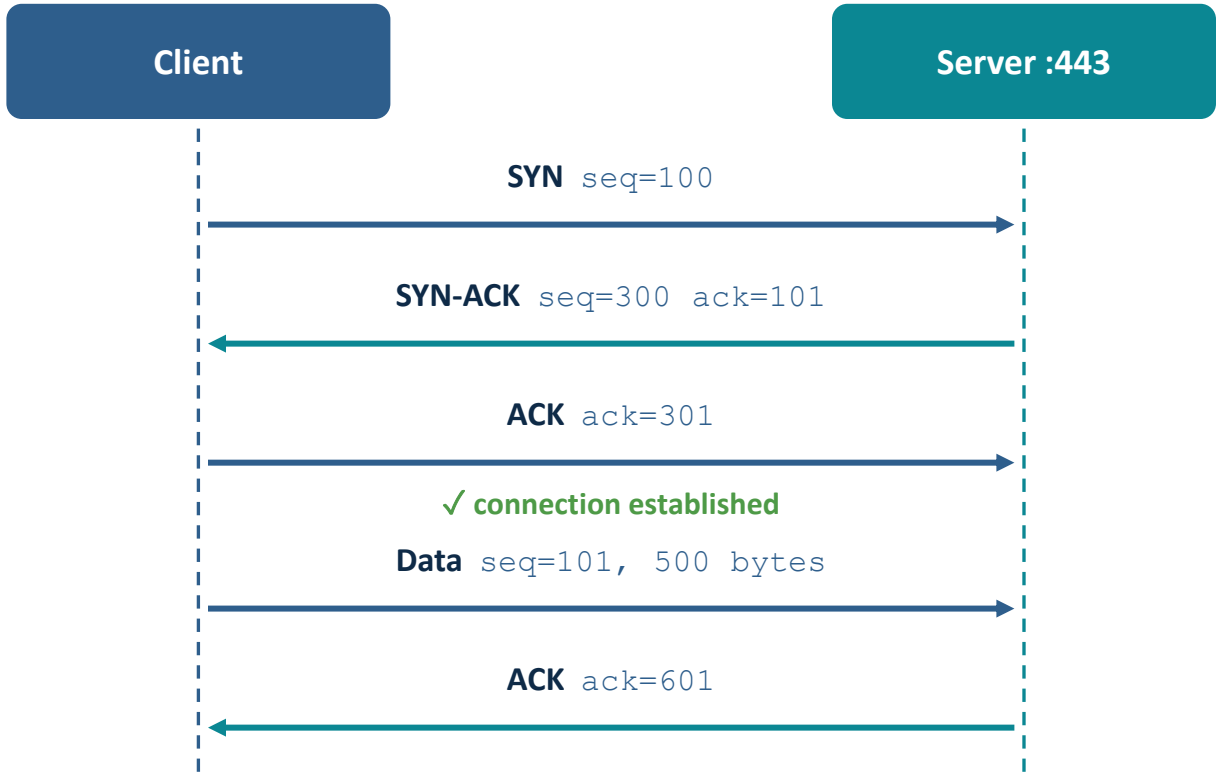


UDP: posting letters
Each one travels alone; no reply expected

Three-way handshake first	**Setup**	None: send immediately
Guaranteed: lost data is resent	**Delivery**	Best effort: may be lost
Bytes arrive in order	**Ordering**	May arrive out of order
Flow and congestion control	**Speed control**	None: the app decides
20-60 bytes	**Header size**	8 bytes
Continuous byte stream	**Data unit**	Independent datagrams
One-to-one only	**Delivery modes**	Unicast, broadcast, multicast

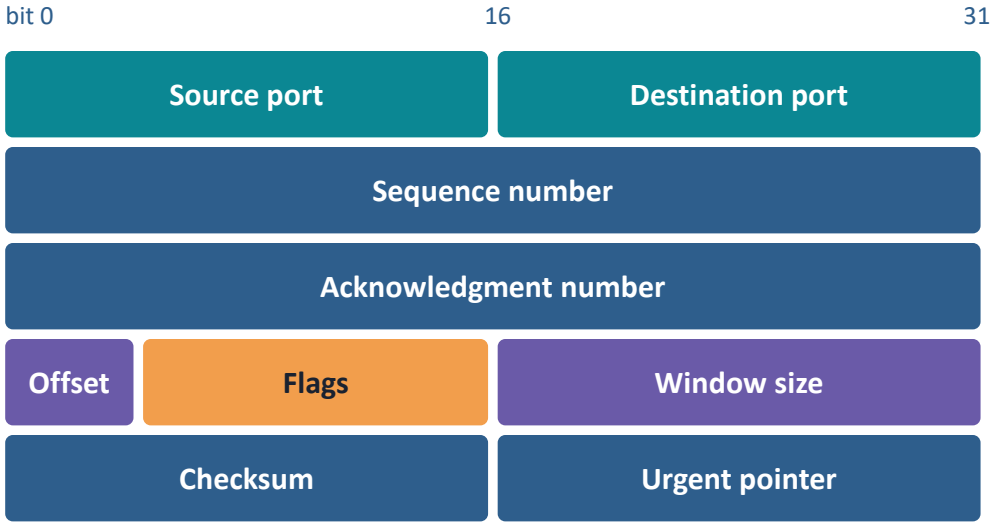
The TCP three-way handshake

Both sides agree on starting sequence numbers before any data flows



Closing: FIN →, ← ACK, ← FIN, ACK → (or RST to abort at once)

TCP header: 20 bytes minimum



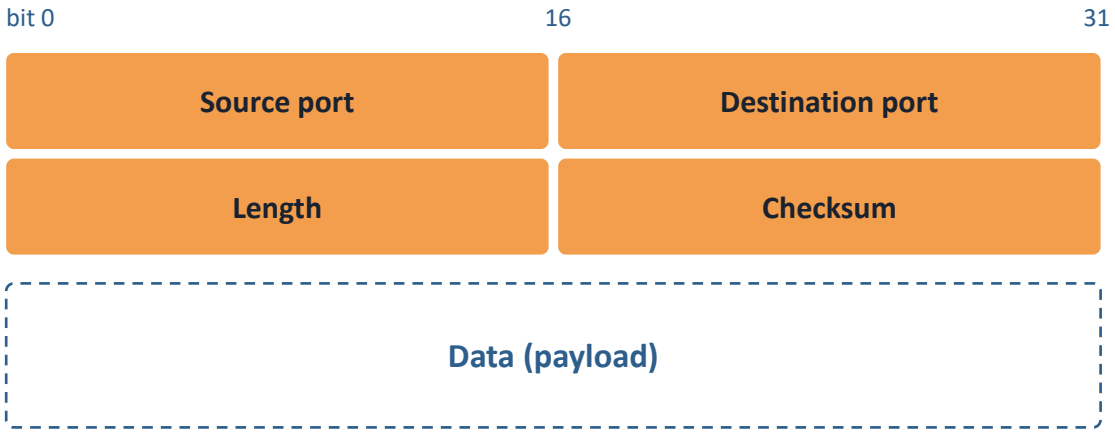
`SYN` open · **`ACK`** acknowledge · **`FIN`** close · **`RST`** abort · **`PSH`** deliver now

Window: how many more bytes the receiver can accept

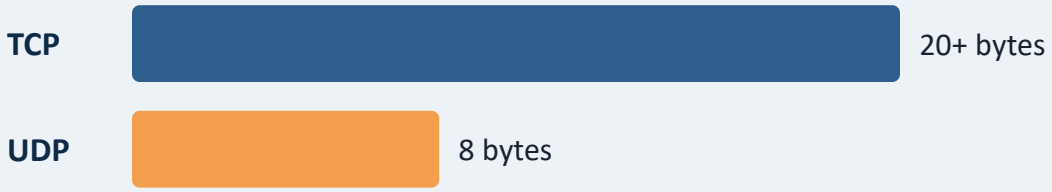
UDP: small, fast and simple

Just ports, a length and a checksum; everything else is up to the application

UDP header: 8 bytes



Header overhead per packet



What UDP gives you, and what it doesn't

No setup
The first packet already carries data

No guarantees
Datagrams can be lost, duplicated or reordered

No flow or congestion control
It sends as fast as the application asks

Message boundaries kept
One send() arrives as one datagram

Broadcast and multicast
One packet to many receivers (DHCP, IPTV)

When to use TCP, when to use UDP

It comes down to one question: is late data still useful?

Choose TCP when...

- Every byte must arrive intact
- Order matters: files, web pages, email
- Waiting is better than losing data

Choose UDP when...

- Late data is useless: voice, video, games
- Exchanges are tiny: DNS, DHCP, NTP
- You need broadcast or multicast

Best of both: QUIC

- Runs on UDP; adds reliability and TLS 1.3
- Setup in 1 round trip (0 when resuming)
- The transport under HTTP/3

Quick check: which would you pick?

File download TCP	**Video call** UDP	**DNS lookup** UDP	**Sending email** TCP	**Online game** UDP	**Web page** TCP or QUIC

Common protocols over TCP and UDP

Well-known services and the transport they normally use

TCP

HTTP / HTTPS	80, 443
SSH	22
SMTP	25, 587
IMAPS	993
FTP	20, 21
BGP	179
MySQL	3306
RDP	3389

UDP

DHCP	67, 68
NTP	123
SNMP	161
Syslog	514
TFTP	69
QUIC / HTTP/3	443
WireGuard VPN	51820
RTP (voice/video)	dynamic

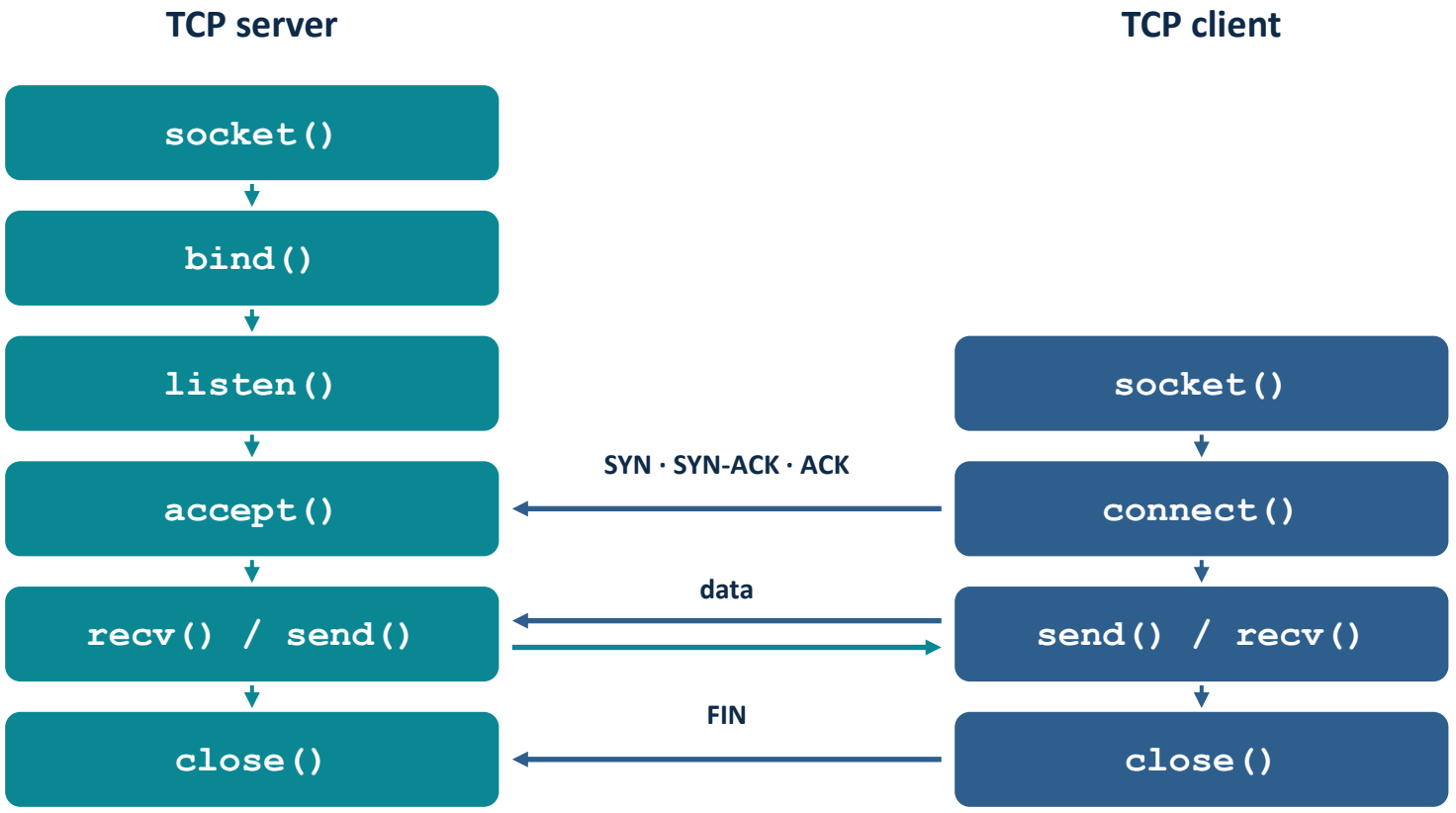
Both

DNS	53
SIP (VoIP)	5060
OpenVPN	1194
Kerberos	88

DNS switches to TCP for large answers and zone transfers. VPNs and SIP prefer UDP but fall back to TCP when UDP is blocked.

The socket API

A socket is one end of a conversation: an IP address and a port behind a file-like handle



accept() returns a new socket for each client; the listening socket keeps waiting for the next one.

UDP: no connection

- **Server:** bind() a port, then loop recvfrom() → sendto()
- **Client:** just sendto() and recvfrom()
- **No** listen(), accept() or connect() needed

Two choices at socket()

- **`AF_INET`** / **`AF_INET6`**: IPv4 or IPv6
- **`SOCK_STREAM`** = TCP · **`SOCK_DGRAM`** = UDP
- Calls block by default; **`settimeout()`** limits the wait

TCP echo server and client

The server waits for connections; the client connects, sends, and reads the reply

tcp_server.py echo server, one client at a time

```python
import socket

HOST, PORT = "0.0.0.0", 5000      # 0.0.0.0 = every interface

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind((HOST, PORT))
    srv.listen()
    print(f"listening on {HOST}:{PORT}")
    while True:
        conn, addr = srv.accept()     # wait for a client
        with conn:
            print("connected:", addr)
            while data := conn.recv(1024):   # b"" = closed
                conn.sendall(data.upper())
            print("closed:", addr)
```

tcp_client.py connect, send two messages, print replies

```python
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.settimeout(5)
    s.connect((HOST, PORT))           # TCP three-way handshake
    for text in ["hello", "sockets in python"]:
        s.sendall(text.encode())      # str -> bytes
        reply = s.recv(1024)
        print(f"{text!r} -> {reply.decode()!r}")
```

`"0.0.0.0"` = listen on every interface

`recv(n)` returns up to n bytes; b"" = closed

`sendall()` loops until every byte is sent

UDP server and client

No connection: every datagram carries its own destination, and the reply may never come

udp_server.py reply to every datagram

```python
import socket

HOST, PORT = "0.0.0.0", 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.bind((HOST, PORT))              # no listen(), no accept()
    print(f"UDP on {HOST}:{PORT}")
    while True:
        data, addr = s.recvfrom(2048)   # one whole datagram
        print(addr, data)
        s.sendto(data.upper(), addr)    # reply to the sender
```

udp_client.py no connection: send and wait for a reply

```python
import socket
import sys

HOST = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.settimeout(2)                     # replies can get lost
    for msg in [b"ping 1", b"ping 2"]:
        s.sendto(msg, (HOST, PORT))
        try:
            data, addr = s.recvfrom(2048)
            print(f"{msg!r} -> {data!r} from {addr}")
        except (TimeoutError, ConnectionError):
            print(f"{msg!r}: no reply (lost or port closed)")
```

`bind()` only: no listen(), no accept()

`recvfrom()` returns the data and the sender

`settimeout()`: replies can be lost